

# 1. ABSTRACT

SecureVote is a robust, localized digital voting system developed to facilitate high-integrity elections within small-scale organizations. Utilizing a Python-based backend with an SQLite3 relational database, the system replaces manual balloting with an automated, encrypted workflow. Key features include **SHA-256 cryptographic hashing** for voter data protection and a **simulated Two-Factor Authentication (2FA)** via session-based OTPs. The system ensures a "One-Voter-One-Vote" policy through rigid database constraints, providing real-time result visualization and an immutable audit trail.

# 2. INTRODUCTION

Institutional governance—ranging from university student councils to corporate boards—requires a voting process that is both transparent and secure. Traditional methods often suffer from "proxy voting" and manual tallying errors. SecureVote bridges this gap by providing a standalone desktop application that ensures the secrecy of the ballot and the accuracy of the result. Unlike web-based systems, this localized approach eliminates external vulnerabilities like DDoS attacks while maintaining industrial-grade security protocols.

### 3. MOTIVATION

The development of **SecureVote** is driven by the need to resolve the "Crisis of Trust" in institutional governance through four core technological pillars:

- **Computational Precision:** Manual counting has a **2-3% error margin**. SecureVote uses an SQL-driven engine to provide 100% accurate, instant results, eliminating human fatigue and bias.
- **Cryptographic Privacy:** Unlike standard forms where admins can see voter choices, SecureVote uses **SHA-256 Hashing**. This "Zero-Knowledge" architecture ensures voter IDs and passwords remain irreversible and unreadable.
- **Fraud Mitigation:** To stop "proxy voting" from unattended workstations, the system implements **Two-Factor Authentication (OTP)**. This ensures the physical user matches the digital identity before a vote is cast.
- **Instant Accountability:** Traditional delays create suspicion. SecureVote provides **Real-Time Analytics** and bar charts, ensuring total transparency the moment the polls close.

### 4. WHY THIS PROJECT?

- **The "Opaque Count" Problem:** Manual systems lack visibility. SecureVote provides instant, graphical results the moment the polls close.
- **Cost-Effective Verification:** Most high-security systems require expensive biometrics. SecureVote uses **OTP-based MFA**, offering high security at zero hardware cost.

- **Institutional Auditing:** It generates a standardized CSV report, allowing for a transparent post-election audit.

## 5. OBJECTIVES AND GOALS

The project aims to establish a **"Trustless" voting environment** where security is enforced by cryptographic code rather than human oversight.

- **Relational Integrity:** Utilizes **SQLite UNIQUE and PRIMARY KEY constraints** to programmatically block "double-voting." Once an ID is flagged, the database rejects further entries, ensuring a strict one-person-one-vote policy.
- **Encryption at Rest:** Maintains data privacy by using **SHA-256 hashing**. Sensitive credentials like passwords and IDs are never stored in plain text, securing the system against both internal and external breaches.
- **Automated Visual Analytics:** Removes human bias from result calculation via a **data-driven dashboard**. It automatically computes vote percentages and identifies winners through real-time bar charts.

## 6. BACKGROUND

The project is architected on the **ACID (Atomicity, Consistency, Isolation, Durability)** properties of relational databases to ensure maximum data reliability.

- **Transactional Security:** By using Python's `sqlite3` module, every vote is treated as a "completed transaction." If a system failure occurs mid-vote, the database performs a **rollback**, ensuring the data remains uncorrupted and consistent.

- **Cryptographic Foundation:** The background logic relies on **One-Way Hash Functions**. By transforming inputs into fixed-size 256-bit digests, the system ensures that voter identity remains anonymous and sensitive information is mathematically irreversible.

## 7. TOOLS & PLATFORM

- To ensure high portability and "zero-dependency" deployment, the following tech stack was utilized:

| Category          | Tool              | Purpose                                                                                  |
|-------------------|-------------------|------------------------------------------------------------------------------------------|
| Logic & Backend   | Python 3.x        | Handles core voting logic, input validation, and secure session management.              |
| User Interface    | Tkinter           | Provides a lightweight, event-driven GUI with a professional "Dark Mode" aesthetic.      |
| Database          | SQLite3           | A serverless RDBMS used for ACID-compliant storage of voter and candidate data.          |
| Cryptography      | Hashlib (SHA-256) | Converts sensitive credentials into irreversible digests to ensure "Encryption at Rest." |
| Identity Security | Random (CSPRNG)   | Generates cryptographically secure 6-digit OTPs for two-factor authentication.           |
| Data Reporting    | CSV Module        | Automates the generation of non-editable audit trails and election results.              |

## 8. HARDWARE REQUIREMENT

| COMPONENT | MINIMUM REQUIREMENT   | RECOMMENDED        |
|-----------|-----------------------|--------------------|
| Processor | 1.6 GHz Dual Core     | 2.0 GHz+ Quad Core |
| RAM       | 2 GB                  | 4 GB               |
| Storage   | 50 MB                 | 100 MB             |
| Display   | 1366 x 768 Resolution | 1920 x 1080 (FHD)  |

## 9. SOFTWARE REQUIREMENT

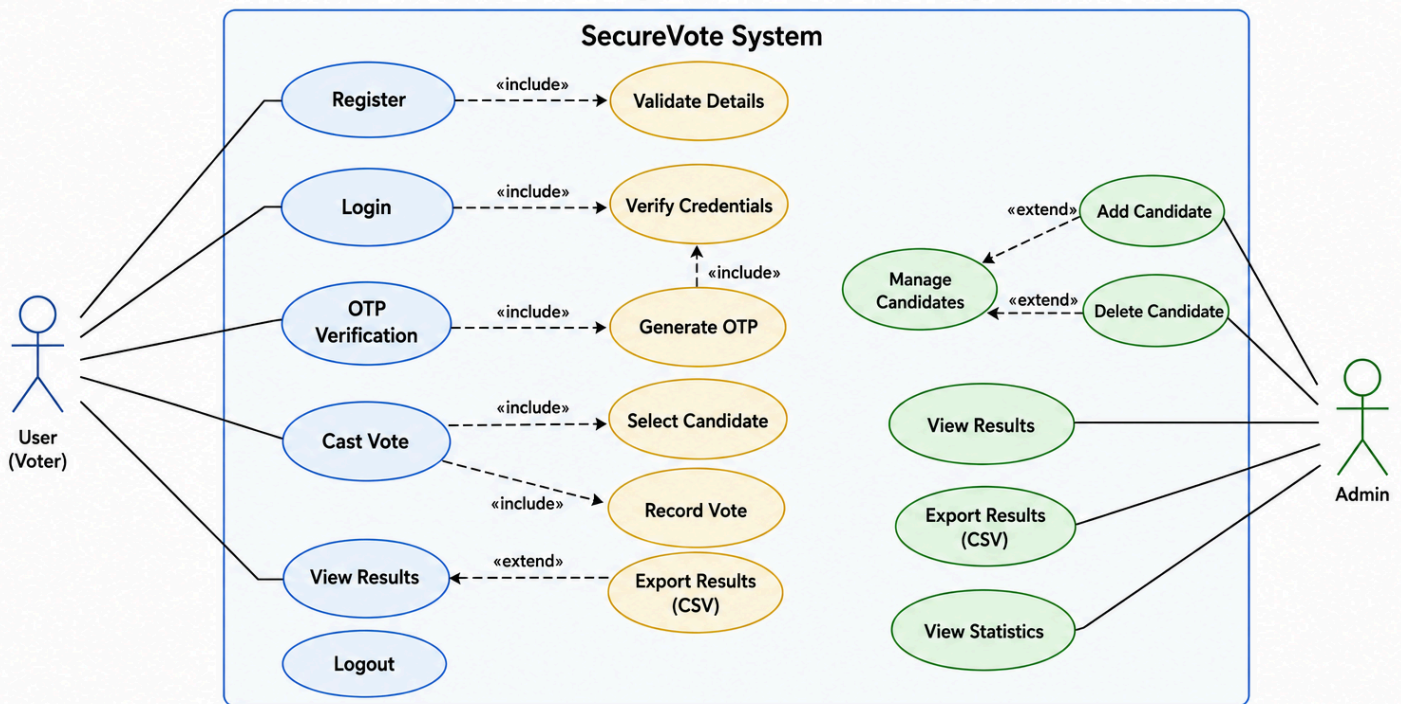
| PLATFORM TYPE    | SPECIFICATION                                  |
|------------------|------------------------------------------------|
| Operating System | Windows 10/11, macOS, or Linux (Ubuntu 20.04+) |
| Python Version   | Python 3.8 or higher                           |
| Development IDE  | VS Code / PyCharm / Sublime Text               |
| Database Tool    | DB Browser for SQLite (for external auditing)  |

# 10. SOFTWARE REQUIREMENTS SPECIFICATIONS DIAGRAMS (SRS)

## A) Unified Modeling Language (UML)Diagram:

The UML Diagram below shows the entire working of the voting system.

UML Use Case Diagram – SecureVote System



- The system has **two actors**:
  - **User (Voter)** – interacts with the system to register, login, verify OTP, cast vote, and view results.
  - **Admin** – manages candidates and monitors the system.

**User Actions** **Register** → creates a new account (includes validation)

- **Login** → verifies credentials
- **OTP Verification** → adds extra security (2FA)
- **Cast Vote** → select candidate and record vote
- **View Results** → see election outcomes
- **Logout** → exit system

### Admin Actions

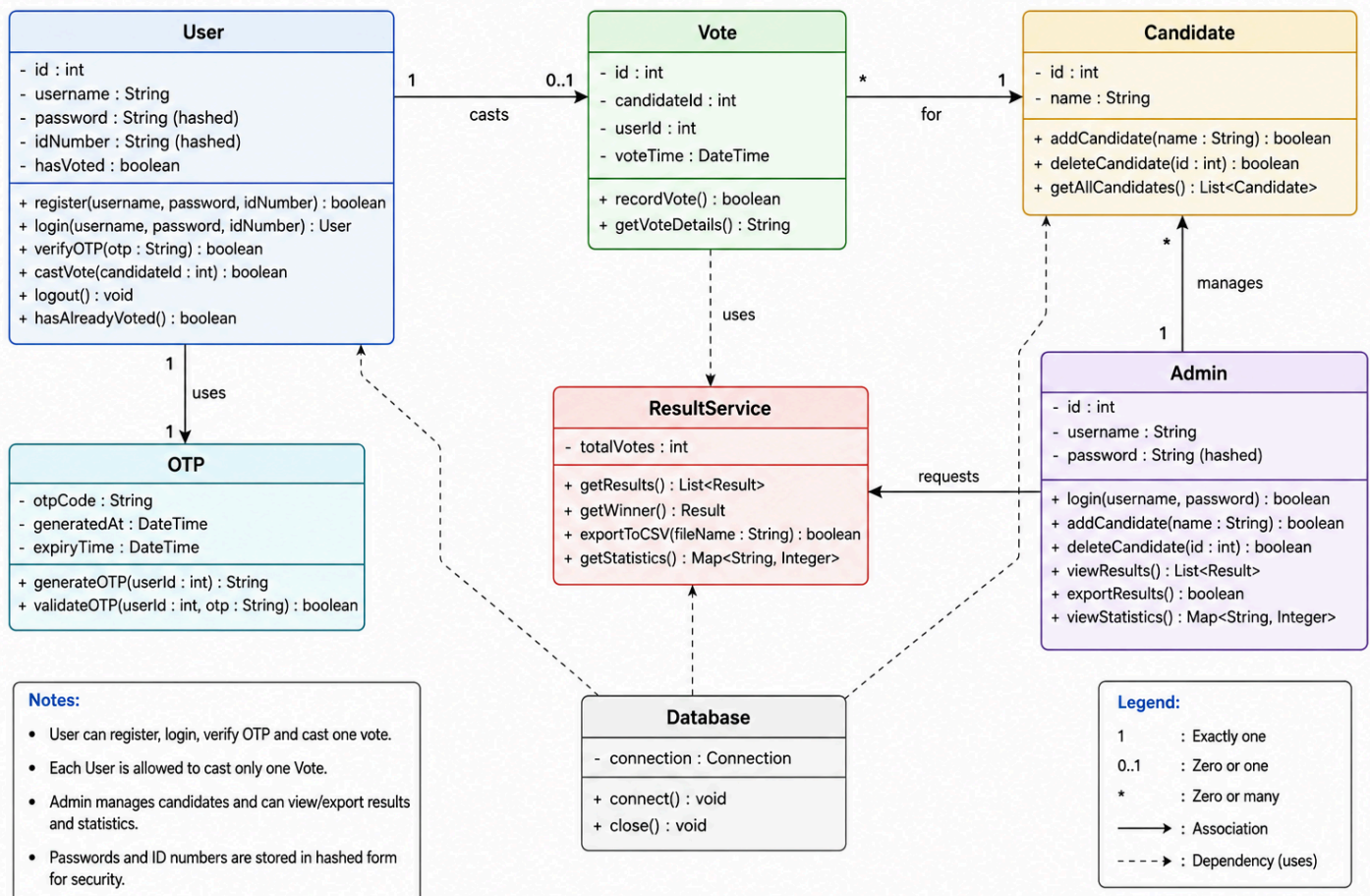
- **Manage Candidates** : Add Candidate Delete Candidate.
- **View Results and Export Results (CSV)**
- **View Statistics**

## Relationships

- «include» → mandatory steps (e.g., login includes verification)
- «extend» → optional actions (e.g., export results)

## B) Class Breakdown

UML Class Diagram – SecureVote System



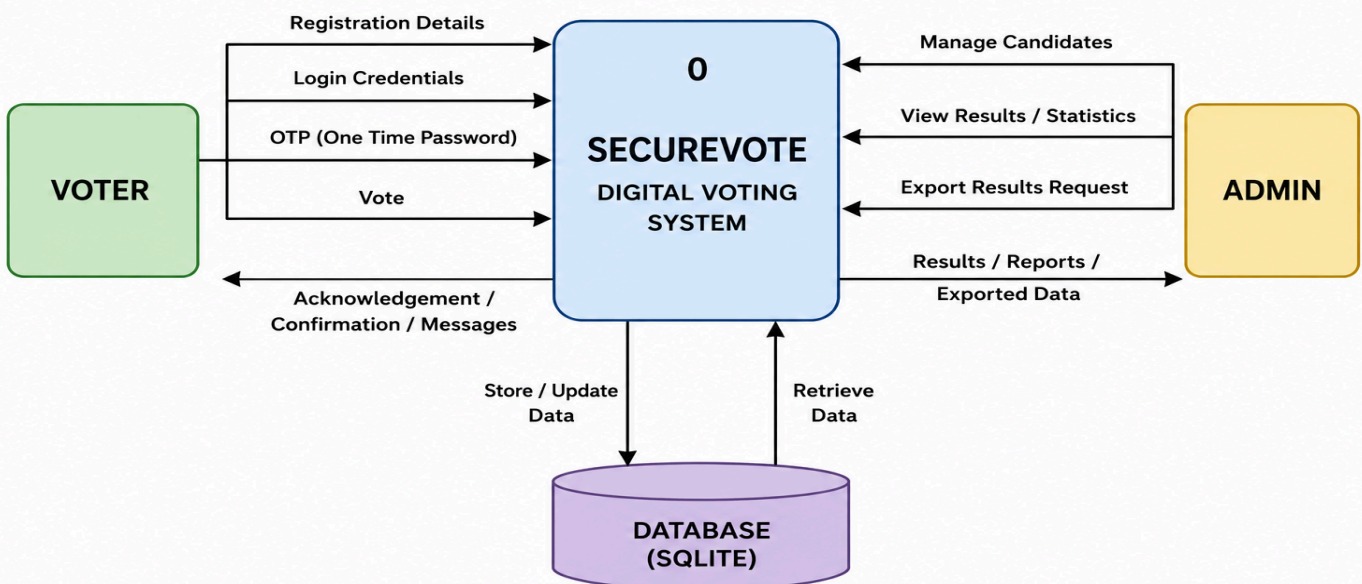
- **User**
  - Stores user details (username, password, ID, voting status)
  - Functions: register, login, verify OTP, cast vote
- **Admin**
  - Manages the system
  - Functions: add/delete candidates, view/export results, view statistics
- **Candidate**
  - Stores candidate info (id, name)
  - Functions: manage candidate list
- **Vote**
  - Represents a vote cast by a user
  - Links User → Candidate
  - Stores vote time and details



- **OTP**
  - Handles security verification
  - Functions: generate and validate OTP
- **ResultService**
  - Processes voting results
  - Functions: get results, winner, export CSV, statistics
- **Database**
  - Manages data storage and connection

## B) Data Flow Diagram (DFD) • Level – 0 DFD

### DATA FLOW DIAGRAM (DFD) LEVEL 0 – CONTEXT DIAGRAM

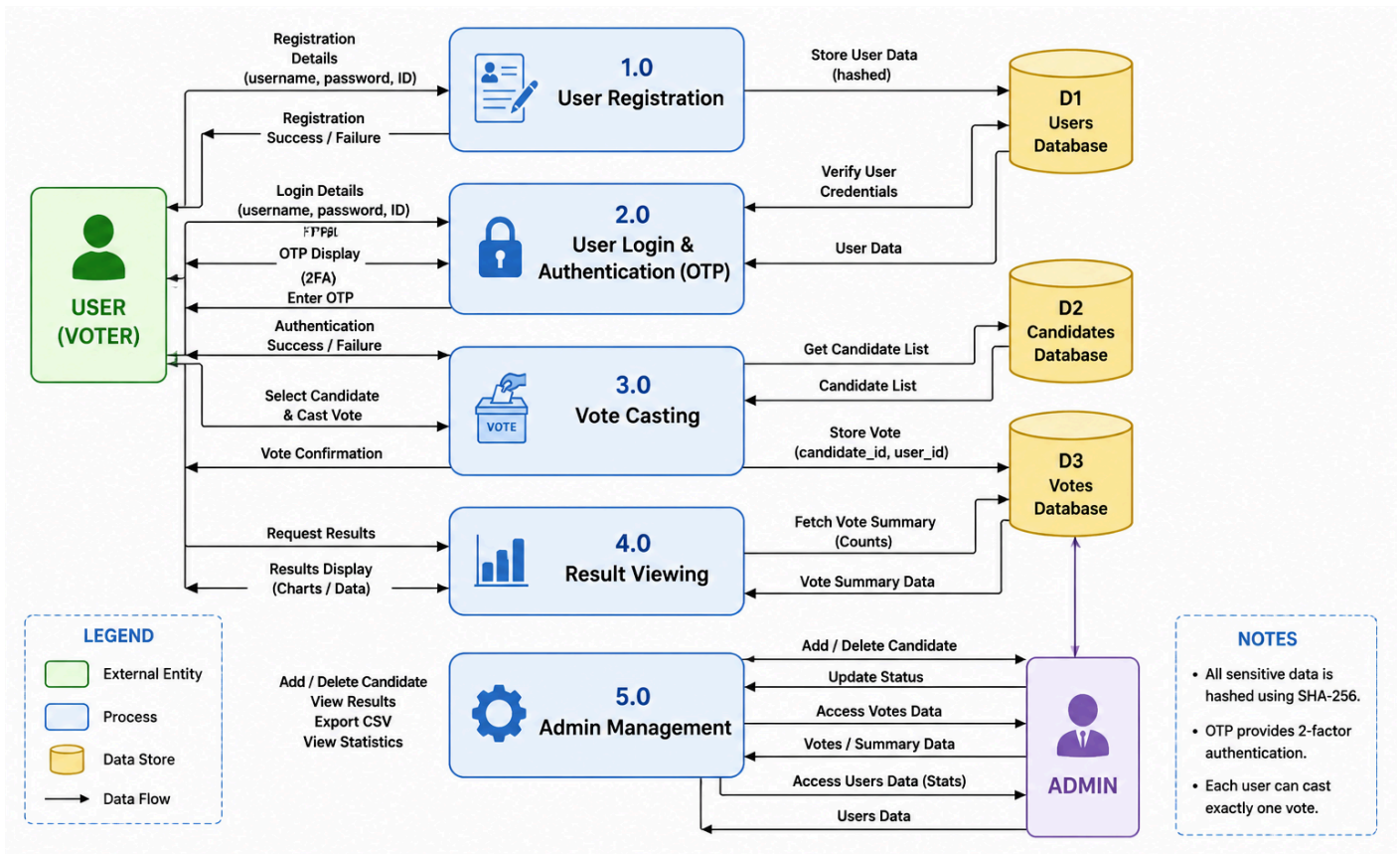


The Level 0 DFD (Context Diagram) defines the scope of the Secure Online Voting System. It represents the entire application as a single process and identifies the data exchange with external entities.

| Entity       | Data Inbound (Input)          | Data Outbound (Output)                |
|--------------|-------------------------------|---------------------------------------|
| <b>Voter</b> | Credentials, OTP, Vote Choice | Auth Status, OTP Prompt, Confirmation |
| <b>Admin</b> | Admin Login, Candidate Info   | Visual Analytics, CSV Audit Trail     |

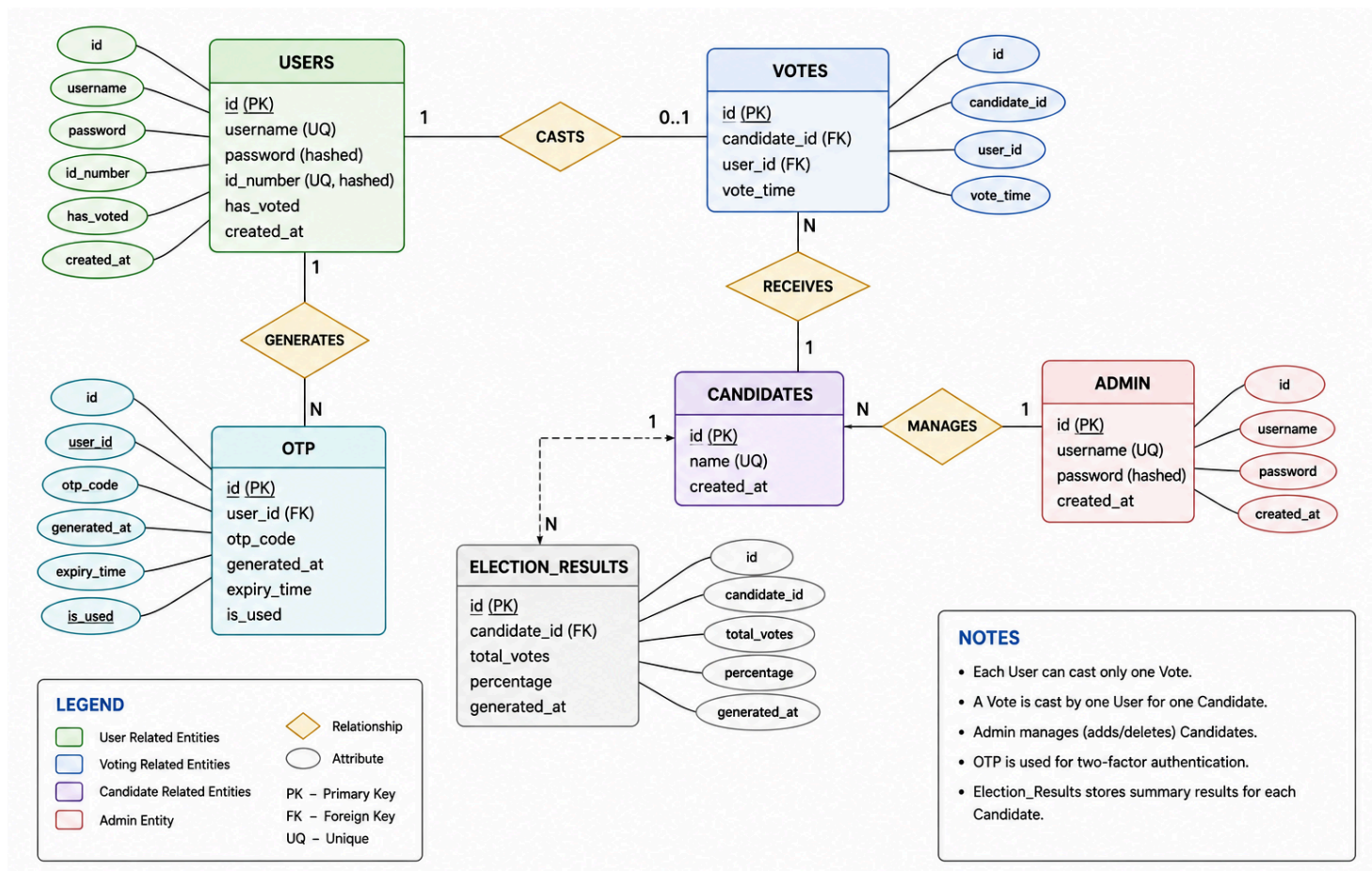


## • Level – 1 DFD



| Process      | Data Input  | Data Output   | Target Store      |
|--------------|-------------|---------------|-------------------|
| Registration | Raw ID/PW   | Hashed Digest | D1: Voters        |
| MFA Logic    | Login Input | Session OTP   | D1 (Status Check) |
| Ballot       | Selection   | Secure Vote   | D3: Votes         |
| Analytics    | Query       | Charts/CSV    | Admin UI          |

## C) Entity-Relationship (E-R) DIAGRAM



### Main Entities

- **USERS**
  - Stores voter details (username, password, ID, voting status)
  - Each user can vote only once
- **CANDIDATES**
  - Stores candidate information (id, name)
- **VOTES**
  - Records each vote
  - Links User → Candidate
- **ADMIN**
  - Manages candidates and system
- **OTP**
  - Stores OTP details for login verification
- **ELECTION\_RESULTS**
  - Stores summarized results (total votes, percentage)

## **Relationships**

- **User → Votes (CASTS)**
  - One user can cast 0 or 1 vote
- **Vote → Candidate (RECEIVES)**
  - Many votes can go to one candidate
- **Admin → Candidate (MANAGES)**
  - Admin can add/delete candidates
- **User → OTP (GENERATES)**
  - User generates OTP for authentication
- **Candidate → Results**
  - Results are calculated per candidate

## **11. APPLICATIONS**

SecureVote is designed for high-integrity environments that require professional security without massive infrastructure.

- **Corporate Governance:** Facilitates "Secret Consensus" for board-level decisions or policy shifts. The SHA-256 anonymity protects voters from professional retaliation, ensuring truly unbiased outcomes.
- **Academic Recognition:** Ideal for university polls like "Best Researcher" or "Student of the Year." The MFA logic ensures only authorized faculty vote, while CSV auditing provides a formal record for institutional archives.
- **Small-Scale Community Polls:** Replaces manual paper ballots in NGOs, Housing Societies, or local clubs. The Real-Time Analytics provide immediate results, eliminating the "window of suspicion" found in manual counting.
- **Cybersecurity Demonstrations:** Serves as a practical tool for teaching "Encryption at Rest" and ACID database properties in technical workshops and CSE educational labs.

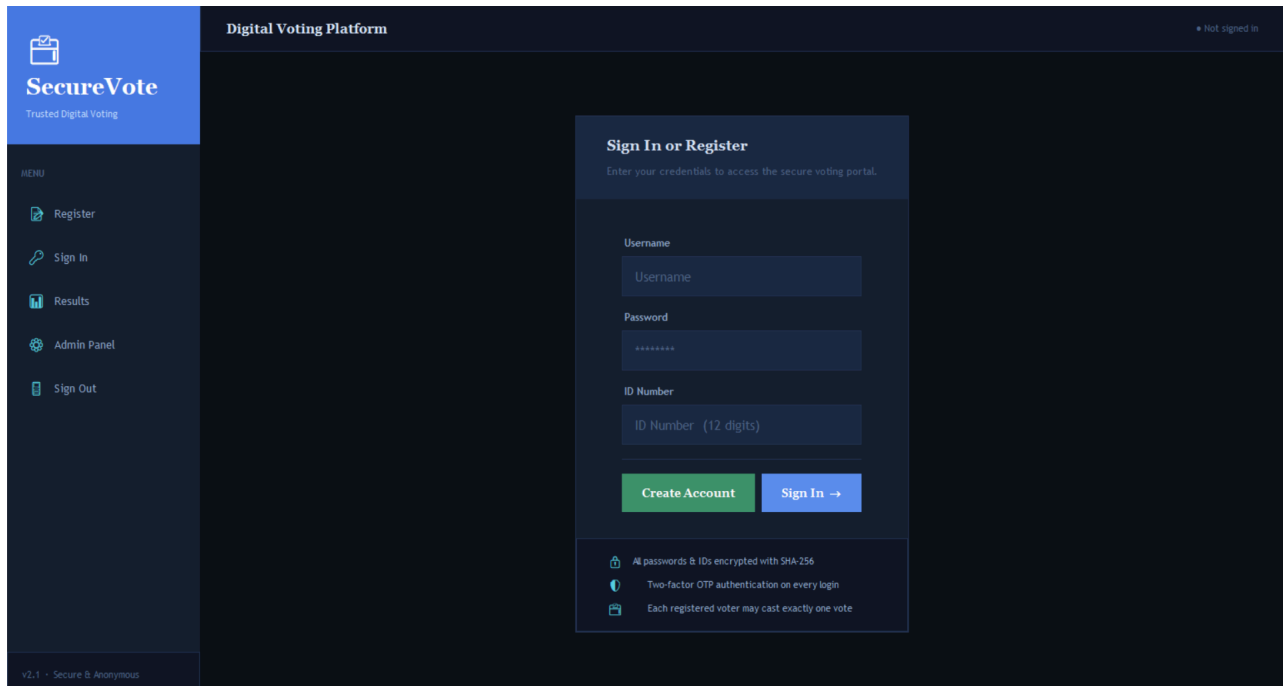
## 12. FUTURE SCOPE & ENHANCEMENTS

The current architecture of SecureVote serves as a secure foundation that can be scaled into a more resilient, industrial-grade platform through the following advancements:

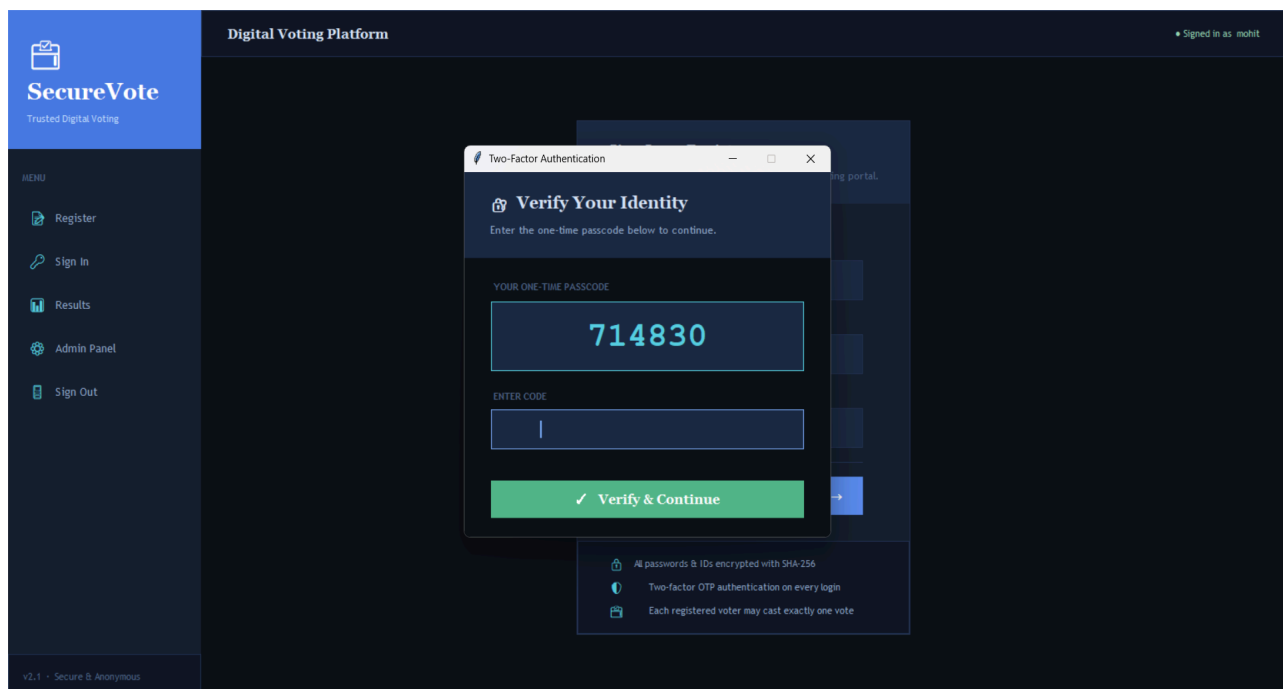
- **Network Synchronization (Client-Server Model):** Transitioning from a standalone desktop application to a LAN-based architecture using Python's `socket` programming or `Flask`. This would allow multiple voting booths (Clients) to communicate with a central secure server, enabling simultaneous voting across an entire campus or office.
- **Blockchain Integration (Immutable Ledger):** To achieve "Absolute Trust," the SQLite backend can be integrated with a Private Blockchain (e.g., Hyperledger or Ethereum). By storing vote hashes on a decentralized ledger, the election becomes 100% tamper-proof, as no single administrator could alter the data. Migrating the local database to AWS RDS or Google Cloud SQL. This ensures high availability and allows administrators to monitor election progress from any location via a secure web-based dashboard.
- **e results without breaking the chain's integrity.**
- **Live Communication APIs:** Integrating Twilio (SMS) or SendGrid (Email) APIs to automate OTP delivery. This moves beyond session-based simulation to real-world verification, ensuring that the voter has physical possession of their registered device or institutional email at the time of polling.
- **Biometric Hardening:** Adding a layer of physical security by integrating Face Recognition (OpenCV) or fingerprint scanning. This would eliminate the risk of "credential sharing" and move the system toward a sophisticated Zero-Trust security model.
- **Cloud Persistence:** Migrating the local database to AWS RDS or Google Cloud SQL. This ensures high availability and allows administrators to monitor election progress from any location via a secure web-based dashboard.

# 13. SCREENSHOTS OF THE PROJECT

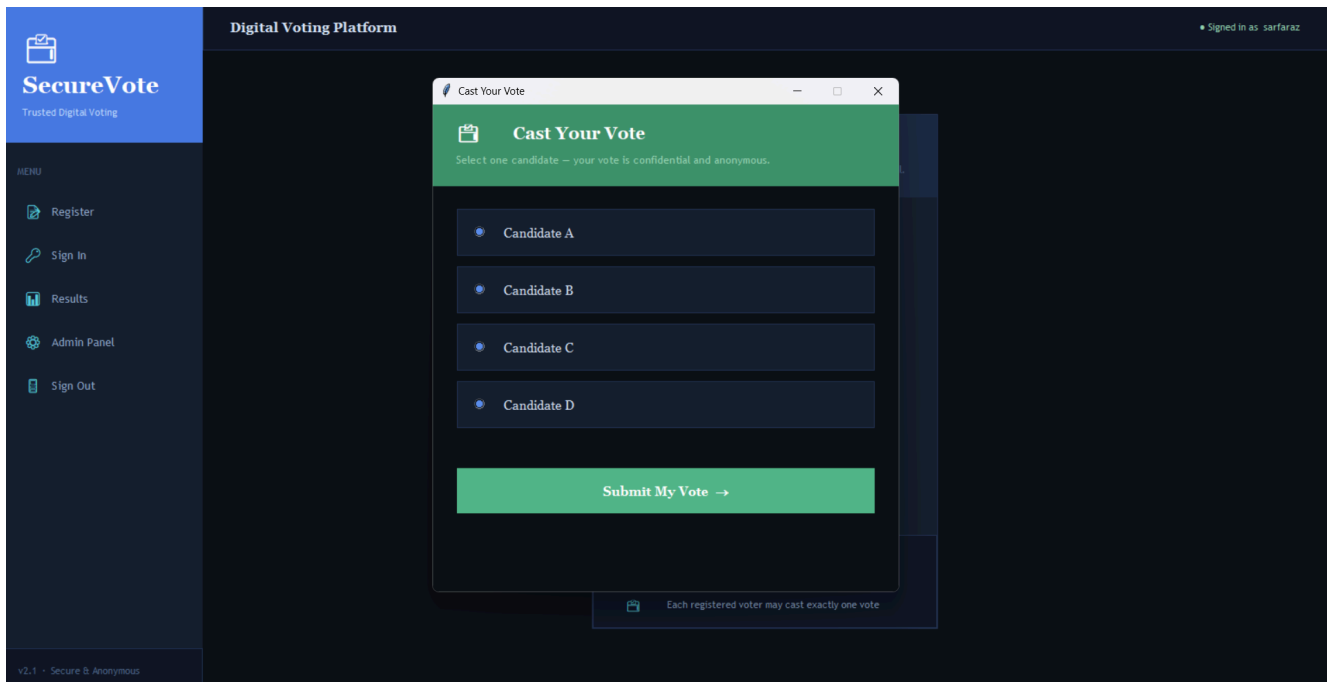
## 1. Auth Card: The main landing page with Georgia font and Dark Mode UI



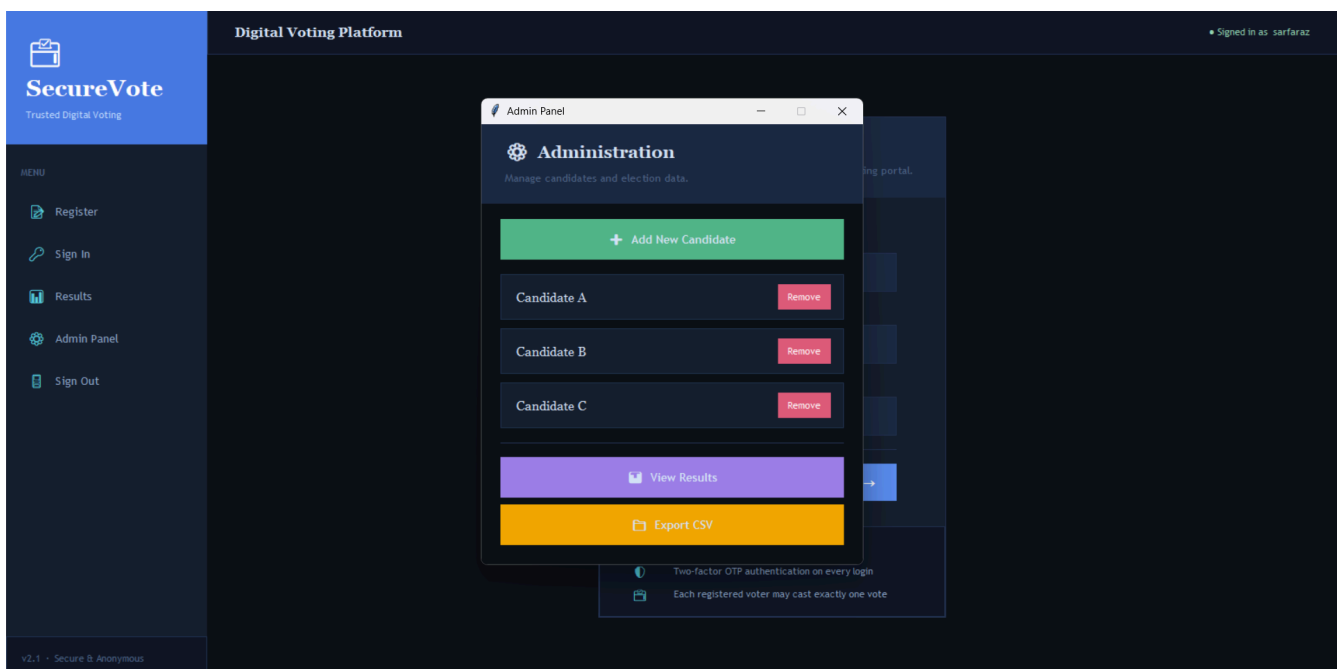
## 2. OTP Dialog: The popup showing the verification code.



### 3. Ballot Window: The list of candidates with custom-styled RadioButtons.

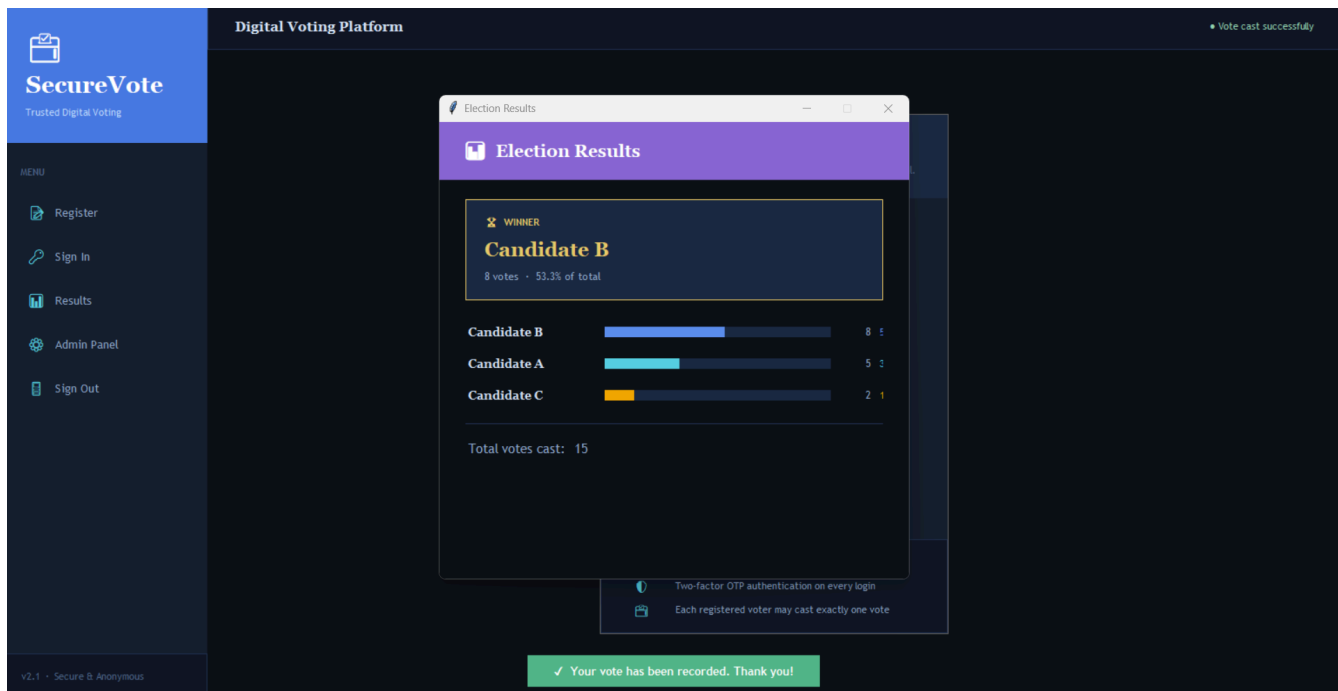


### 4. Admin Panel: Showing candidate management and the "Export CSV" button.





## 5. Analytics Screen: The Bar Chart showing Candidates and the winner.



## 14. REFERENCE

### Books

- Software Engineering: A Practitioner's Approach – Used for system design and architecture
- Database System Concepts – Used for database design and ER diagrams
- Python Programming: An Introduction to Computer Science – Used for Python basics and GUI logic

### Online Resources

- Python Documentation – For syntax and libraries
- Tkinter Documentation – For GUI development
- SQLite Website – For database handling
- W3Schools – For basic concepts
- GeeksforGeeks – For examples and advanced topics

## **Security Concepts**

- SHA-256 – For password and ID encryption
- Two-Factor Authentication – For OTP verification

## **Design & Modeling**

- UML – For system diagrams
- DFD – For data flow representation
- ER Model – For database design